Kubernetes ConfigMaps & Secrets — Complete Study Notes

CKA/CKAD Exam Preparation | Basic to Expert Level

Table of Contents

1. Introduction & Purpose
2. ConfigMaps — Deep Dive
3. Secrets — Deep Dive
4. Security Best Practices
5. Hands-On Examples
6. Real-World Patterns
7. CKA/CKAD Exam Tips

---

1. Introduction & Purpose

1.1 What Are ConfigMaps and Secrets?

ConfigMaps and **Secrets** are Kubernetes API objects that decouple **configuration data** from **container images**. They allow you to externalize configuration so that your application containers remain portable across environments.

```
┌──────────────────────────────────────────────────────────────┐
│                    Kubernetes Cluster                        │
│                                                              │
│  ┌──────────────┐    ┌──────────────┐    ┌──────────────┐   │
│  │  ConfigMap   │    │   Secret     │    │   Pod        │   │
│  │              │    │              │    │              │   │
│  │ DB_HOST=...  │───>│ DB_PASS=...  │───>│ App Container│   │
│  │ DB_PORT=...  │    │ API_KEY=...  │    │  reads config│   │
│  │ LOG_LVL=...  │    │ TLS_CERT=..  │    │  at runtime  │   │
│  └──────────────┘    └──────────────┘    └──────────────┘   │
│                                                              │
└──────────────────────────────────────────────────────────────┘
```

Feature	ConfigMap	Secret
Purpose	Non-sensitive configuration data	Sensitive data (passwords, tokens, keys)
Encoding	Plain text	Base64-encoded (at rest)
Size Limit	1 MiB	1 MiB
Encryption	Not encrypted	Can be encrypted at rest (optional)
tmpfs	No (stored on disk in volumes)	Yes (stored in tmpfs in memory when mounted as volumes)
RBAC	Standard RBAC	Should have stricter RBAC

1.2 Why Separate Configuration from Application Code?

The **12-Factor App** methodology (Factor III) mandates storing configuration in the environment, not in code. Benefits:

1. **Portability** — Same container image works in dev, staging, and prod
2. **Security** — Secrets are not baked into images or committed to Git
3. **Flexibility** — Change configuration without rebuilding images
4. **Separation of Concerns** — Developers focus on code, operators manage config
5. **Auditability** — Kubernetes tracks changes to ConfigMaps and Secrets via the API server

1.3 ConfigMaps vs Secrets — When to Use Which

```
Decision Tree:

                    ┌──────────────────────┐
                    │ Is the data sensitive? │
                    └──────────┬───────────┘
                               │
                    ┌──────────┴──────────┐
                    │                     │
                   YES                    NO
                    │                     │
              ┌─────┴─────┐         ┌─────┴─────┐
              │  Secret   │         │  ConfigMap  │
              └───────────┘         └─────────────┘


Examples:
  ConfigMap: database hostnames, port numbers, log levels, feature flags,
             application.properties, nginx.conf, config.yaml files

  Secret:    database passwords, API keys, TLS certificates/keys,
             OAuth tokens, SSH keys, docker registry credentials
```

> ***IMPORTANT (CKA/CKAD):*** *The existing files in this project show an API key stored in a ConfigMap — this is an* ***anti-pattern****. API keys should ALWAYS be stored in Secrets, not ConfigMaps.*

---

2. ConfigMaps — Deep Dive

2.1 ConfigMap Architecture

```
┌──────────────────────────────────────────────────────────┐
│                    ConfigMap Object                      │
│                                                          │
│  apiVersion: v1                                          │
│  kind: ConfigMap                                         │
│  metadata:                                               │
│    name: my-config                                       │
│    namespace: default                                    │
│                                                          │
│  ┌──────────────────────┐  ┌──────────────────────────┐  │
│  │   data: {}           │  │   binaryData: {}         │  │
│  │   (UTF-8 strings)    │  │   (base64-encoded binary)│  │
│  │                      │  │                          │  │
│  │  key1: "value1"      │  │  logo.png: "iVBOR..."    │  │
│  │  key2: "value2"      │  │                          │  │
│  │  config.yaml: |      │  │                          │  │
│  │    server:           │  │                          │  │
│  │      port: 8080      │  │                          │  │
│  └──────────────────────┘  └──────────────────────────┘  │
│                                                          │
│  immutable: false  (optional, default)                   │
│                                                          │
└──────────────────────────────────────────────────────────┘
```

2.2 Creating ConfigMaps

Method 1: From Literal Values

```
# Single literal
kubectl create configmap app-config --from-literal=APP_ENV=production

# Multiple literals
kubectl create configmap app-config \
  --from-literal=APP_ENV=production \
  --from-literal=APP_PORT=8080 \
  --from-literal=LOG_LEVEL=info

# Verify
kubectl get configmap app-config -o yaml
```

Method 2: From a Single File

```
# Create a config file
cat <<EOF > application.properties
server.port=8080
database.host=postgres.default.svc.cluster.local
database.port=5432
database.name=myapp
logging.level=INFO
EOF

# Create ConfigMap from file (key = filename, value = file content)
kubectl create configmap app-properties --from-file=application.properties

# Create with a custom key name
kubectl create configmap app-properties --from-file=app.conf=application.properties
```

Method 3: From a Directory

```
# All valid files in the directory become keys
mkdir config-dir
echo "value1" > config-dir/key1
echo "value2" > config-dir/key2
echo "value3" > config-dir/key3

kubectl create configmap dir-config --from-file=config-dir/
# Result: keys are "key1", "key2", "key3"
```

Method 4: From an .env File

```
# Create .env file
cat <<EOF > app.env
DB_HOST=postgres.default.svc.cluster.local
DB_PORT=5432
DB_NAME=myapp
CACHE_TTL=300
EOF

# Create ConfigMap from env file
kubectl create configmap env-config --from-env-file=app.env

# Multiple env files
kubectl create configmap multi-env-config \
  --from-env-file=app.env \
  --from-env-file=feature-flags.env
```

> ***Key Difference:*** *`--from-file` creates one key (filename) with the whole content as value. `--from-env-file` parses each `KEY=VALUE` line as a separate key-value pair.*

Method 5: From a YAML Manifest (Declarative)

```yaml
# configmap-declarative.yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
  namespace: default
  labels:
    app: myapp
    environment: production
data:
  # Simple key-value pairs
  APP_ENV: "production"
  APP_PORT: "8080"
  LOG_LEVEL: "info"
  CACHE_ENABLED: "true"

  # Multi-line configuration file
  application.yaml: |
    server:
      port: 8080
      host: 0.0.0.0
    database:
      host: postgres.default.svc.cluster.local
      port: 5432
      name: myapp
      pool:
        min: 5
        max: 20
    logging:
      level: info
      format: json

  # Another config file
  nginx.conf: |
    server {
        listen 80;
        server_name localhost;
        location / {
            proxy_pass http://localhost:8080;
            proxy_set_header Host $host;
            proxy_set_header X-Real-IP $remote_addr;
        }
    }
```

```
kubectl apply -f configmap-declarative.yaml
```

Method 6: Mixed Sources

```
# Combine literals, files, and env files
kubectl create configmap mixed-config \
  --from-literal=EXTRA_KEY=extra_value \
  --from-file=app-config.yaml \
  --from-env-file=defaults.env
```

2.3 Consuming ConfigMaps

Method A: As Individual Environment Variables

```
apiVersion: v1
kind: Pod
metadata:
  name: app-pod
spec:
  containers:
  - name: app
    image: myapp:1.0
    env:
    # Pick specific keys from ConfigMap
    - name: DATABASE_HOST        # Env var name in container
      valueFrom:
        configMapKeyRef:
          name: app-config       # ConfigMap name
          key: DB_HOST           # Key in ConfigMap
    - name: DATABASE_PORT
      valueFrom:
        configMapKeyRef:
          name: app-config
          key: DB_PORT
    - name: LOG_LEVEL
      valueFrom:
        configMapKeyRef:
          name: app-config
          key: LOG_LEVEL
          optional: true         # Pod starts even if key is missing
```

Method B: All Keys as Environment Variables (envFrom)

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: app-pod
spec:
  containers:
  - name: app
    image: myapp:1.0
    envFrom:
    # Import ALL key-value pairs as env vars
    - configMapRef:
        name: app-config
      prefix: APP_              # Optional: prefix all keys with APP_
    - configMapRef:
        name: db-config
      prefix: DB_
```

> ***Note:*** *Keys that are not valid environment variable names (e.g., containing* `-` *or* `.`*) are skipped when using* `envFrom`*.*

Method C: As Command-Line Arguments

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: app-pod
spec:
  containers:
  - name: app
    image: myapp:1.0
    env:
    - name: LOG_LEVEL
      valueFrom:
        configMapKeyRef:
          name: app-config
          key: LOG_LEVEL
    command: ["/bin/sh"]
    args: ["-c", "myapp --log-level=$(LOG_LEVEL) --verbose"]
```

Method D: As Mounted Volumes (Files)

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: app-pod
spec:
  containers:
  - name: app
    image: nginx:1.25
    volumeMounts:
    - name: config-volume
      mountPath: /etc/nginx/conf.d   # Directory where files appear
      readOnly: true
  volumes:
  - name: config-volume
    configMap:
      name: nginx-config             # Each key becomes a file
      # Optional: set file permissions
      defaultMode: 0644
```

Method D.1: Mount Specific Keys Only

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: app-pod
spec:
  containers:
  - name: app
    image: nginx:1.25
    volumeMounts:
    - name: config-volume
      mountPath: /etc/nginx/conf.d
  volumes:
  - name: config-volume
    configMap:
      name: nginx-config
      items:                          # Only mount specific keys
      - key: nginx.conf              # Key from ConfigMap
        path: default.conf           # Filename in the volume
      - key: mime.types
        path: mime.types
```

Method D.2: Mount as SubPath (Without Overwriting Directory)

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: app-pod
spec:
  containers:
  - name: app
    image: myapp:1.0
    volumeMounts:
    - name: config-volume
      mountPath: /app/config/application.yaml  # Single file path
      subPath: application.yaml                # Key from ConfigMap
  volumes:
  - name: config-volume
    configMap:
      name: app-config
```

> ***Warning:*** *When using `subPath`, the file does NOT receive automatic updates when the ConfigMap changes. This is an important CKA/CKAD exam gotcha.*

2.4 ConfigMap Consumption Methods — Comparison

Method	Auto-Update?	Restart Needed?	Use Case
Env Vars (env/envFrom)	NO	YES (to pick up new values)	Simple key-value config
Volume Mount (no subPath)	YES (~60-90s delay)	NO (kubelet syncs files)	Config files (nginx, etc.)
Volume Mount (with subPath)	NO	YES	Single file injection
Command Args	NO	YES	Startup flags

2.5 Updating ConfigMaps and Pod Behavior

```
# Edit ConfigMap directly
kubectl edit configmap app-config

# Replace from updated file
kubectl create configmap app-config --from-file=app.conf --dry-run=client -o yaml \
  | kubectl apply -f -

# Patch a single key
kubectl patch configmap app-config --type merge \
  -p '{"data":{"LOG_LEVEL":"debug"}}'
```

Update propagation for volume-mounted ConfigMaps:

```
ConfigMap Updated (kubectl apply)
        |
        ▼ (kubelet sync period: ~60-90 seconds)
   Mounted files updated atomically
   (via symlink swap)
        |
        ▼
   Application must detect file change
   (inotify, polling, or SIGHUP)
```

> ***Key Point:*** *The kubelet checks whether the mounted ConfigMap is fresh on every periodic sync. The total delay from the moment a ConfigMap is updated to the new keys being projected to the Pod can be as long as the kubelet sync period + cache propagation delay.*

2.6 Immutable ConfigMaps

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: immutable-app-config
data:
  APP_ENV: "production"
  DB_HOST: "prod-db.example.com"
immutable: true    # <-- Cannot be changed after creation
```

Benefits of Immutable ConfigMaps:

1. **Performance** — Kubelet stops watching for changes, reducing API server load
2. **Safety** — Prevents accidental changes to critical production config
3. **Scale** — Significantly reduces load on the kube-apiserver for clusters with tens of thousands of ConfigMaps

To update an immutable ConfigMap: Delete and recreate it (pods referencing it must be restarted).

```
# Cannot edit — this will fail:
kubectl edit configmap immutable-app-config  # ERROR

# Must delete and recreate:
kubectl delete configmap immutable-app-config
kubectl apply -f updated-configmap.yaml
# Then restart pods:
kubectl rollout restart deployment/my-app
```

2.7 Size Limits and Constraints

Constraint	Limit
Maximum data size	**1 MiB** (1,048,576 bytes)
Key naming	Must be valid DNS subdomain (alphanumeric, `-`, `_`, `.`)
Namespace scoped	ConfigMap must be in the same namespace as the Pod
Binary data	Use `binaryData` field (base64-encoded)
Empty values	Allowed (key with empty string value)

---

3. Secrets — Deep Dive

3.1 Secret Types

```
┌──────────────────────────────────┬──────────────────────────────────────┐
│           Secret Type            │              Purpose                 │
├──────────────────────────────────┼──────────────────────────────────────┤
│ Opaque                           │ Generic, user-defined data (default) │
│ kubernetes.io/tls                │ TLS certificate + private key        │
│ kubernetes.io/dockerconfigjson   │ Docker registry credentials          │
│ kubernetes.io/dockercfg          │ Docker registry (legacy format)      │
│ kubernetes.io/basic-auth         │ Basic authentication (user + password) │
│ kubernetes.io/ssh-auth           │ SSH private key authentication       │
│ kubernetes.io/service-account-token│ ServiceAccount token (auto-created) │
│ bootstrap.kubernetes.io/token    │ Bootstrap token for node joining     │
└──────────────────────────────────┴──────────────────────────────────────┘
```

3.2 Creating Secrets

Method 1: From Literals (Opaque)

```
# Create a generic/opaque secret
kubectl create secret generic db-credentials \
  --from-literal=username=admin \
  --from-literal=password='S3cur3P@ss!'

# Verify (data is base64-encoded)
kubectl get secret db-credentials -o yaml
```

Method 2: From Files

```
# Create files with sensitive data
echo -n 'admin' > ./username.txt
echo -n 'S3cur3P@ss!' > ./password.txt

# Create secret from files
kubectl create secret generic db-credentials \
  --from-file=username=./username.txt \
  --from-file=password=./password.txt

# Clean up local files!
rm ./username.txt ./password.txt
```

> ***Tip:*** *Use `echo -n` (no trailing newline) to avoid accidental newlines in your secrets.*

Method 3: TLS Secret

```
# Create TLS secret from certificate and key files
kubectl create secret tls my-tls-secret \
  --cert=path/to/tls.crt \
  --key=path/to/tls.key
```

Equivalent YAML:

```
apiVersion: v1
kind: Secret
metadata:
  name: my-tls-secret
type: kubernetes.io/tls
data:
  tls.crt: <base64-encoded-certificate>
  tls.key: <base64-encoded-private-key>
```

Method 4: Docker Registry Secret

```
kubectl create secret docker-registry my-registry-secret \
  --docker-server=my-registry.example.com:5000 \
  --docker-username=myuser \
  --docker-password=mypassword \
  --docker-email=user@example.com
```

Usage in Pod:

```
apiVersion: v1
kind: Pod
metadata:
  name: private-image-pod
spec:
  imagePullSecrets:
  - name: my-registry-secret
  containers:
  - name: app
    image: my-registry.example.com:5000/myapp:latest
```

Method 5: Basic Auth Secret

```
apiVersion: v1
kind: Secret
metadata:
  name: basic-auth-secret
type: kubernetes.io/basic-auth
stringData:                     # <-- plain text, auto-encoded by K8s
  username: admin
  password: t0p-Secret
```

Method 6: SSH Auth Secret

```
kubectl create secret generic ssh-key-secret \
  --from-file=ssh-privatekey=$HOME/.ssh/id_rsa \
  --type=kubernetes.io/ssh-auth
```

3.3 Base64 Encoding vs stringData

```
┌──────────────────────────────────────────────────────────────┐
│                     Secret YAML                              │
│                                                              │
│  Option A: data (base64-encoded)                             │
│  ────────────────────────────────                            │
│  data:                                                       │
│    username: YWRtaW4=           # echo -n "admin" | base64   │
│    password: UzNjdXIzUEBzcyE=  # echo -n "S3cur3P@ss!" |base64│
│                                                              │
│  Option B: stringData (plain text — auto-encoded)            │
│  ────────────────────────────────────────────                │
│  stringData:                                                 │
│    username: admin                                           │
│    password: S3cur3P@ss!                                     │
│                                                              │
│  Notes:                                                      │
│  • stringData is write-only (never appears in kubectl get)   │
│  • When both data and stringData have the same key,          │
│    stringData takes precedence                               │
│  • stringData is converted to base64 and stored in data      │
└──────────────────────────────────────────────────────────────┘
```

Encoding/Decoding commands (CKA/CKAD essential):

```
# Encode
echo -n 'my-secret-value' | base64
# Output: bXktc2VjcmV0LXZhbHVl

# Decode
echo 'bXktc2VjcmV0LXZhbHVl' | base64 --decode
# Output: my-secret-value

# Quick decode from kubectl
kubectl get secret db-credentials -o jsonpath='{.data.password}' | base64 --decode
```

3.4 Consuming Secrets

As Environment Variables

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: app-pod
spec:
  containers:
  - name: app
    image: myapp:1.0
    env:
    # Individual keys
    - name: DB_USERNAME
      valueFrom:
        secretKeyRef:
          name: db-credentials
          key: username
    - name: DB_PASSWORD
      valueFrom:
        secretKeyRef:
          name: db-credentials
          key: password
          optional: false        # Pod fails if secret/key missing
```

All Keys via envFrom

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: app-pod
spec:
  containers:
  - name: app
    image: myapp:1.0
    envFrom:
    - secretRef:
        name: db-credentials
      prefix: DB_               # Optional prefix
```

As Mounted Volumes

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: app-pod
spec:
  containers:
  - name: app
    image: myapp:1.0
    volumeMounts:
    - name: secret-volume
      mountPath: /etc/secrets    # Each key = file in this directory
      readOnly: true
  volumes:
  - name: secret-volume
    secret:
      secretName: db-credentials
      defaultMode: 0400          # Read-only for owner (more secure)
      items:                     # Optional: mount specific keys
      - key: password
        path: db-password        # File: /etc/secrets/db-password
```

> ***Security Note:*** *When Secrets are mounted as volumes, they are stored in* ***tmpfs*** *(in-memory filesystem) — they are never written to disk on the node.*

3.5 Secret Rotation and Update Behavior

```
Secret Updated
     │
     ├──> Env Vars: NOT updated (requires pod restart)
     │
     └──> Volume Mount: Updated after kubelet sync (~60-90s)
          BUT NOT if using subPath
```

Best practices for secret rotation:

```
# Strategy 1: Rolling update via annotation change
kubectl patch deployment myapp \
  -p '{"spec":{"template":{"metadata":{"annotations":{"secret-version":"v2"}}}}}'

# Strategy 2: Create new secret with version suffix
kubectl create secret generic db-credentials-v2 \
  --from-literal=password='NewP@ssw0rd!'
# Update deployment to reference db-credentials-v2

# Strategy 3: Use Reloader (open-source tool)
# Reloader watches for ConfigMap/Secret changes and triggers rolling updates
# https://github.com/stakater/Reloader
```

3.6 Immutable Secrets

```
apiVersion: v1
kind: Secret
metadata:
  name: immutable-db-creds
type: Opaque
data:
  password: UzNjdXIzUEBzcyE=
immutable: true
```

Same benefits as immutable ConfigMaps — reduced API server load, prevents accidental modification.

---

4. Security Best Practices

4.1 Why ConfigMaps Should NOT Store Sensitive Data

```
⚠  ANTI-PATTERN (from this project's existing files):

apiVersion: v1
kind: ConfigMap                    # <-- WRONG! Should be a Secret
metadata:
  name: openai-api-config
data:
  OPEN_AI_KEY: sk-Shdn30x=...     # <-- API key in plain text ConfigMap!


 CORRECT APPROACH:

apiVersion: v1
kind: Secret
metadata:
  name: openai-api-secret
type: Opaque
stringData:
  OPEN_AI_KEY: sk-Shdn30x=...     # Stored base64-encoded, can be encrypted at rest
```

Why this matters:

1. ConfigMaps are visible to anyone with `get` permissions on ConfigMaps
2. ConfigMaps are not encrypted at rest by default
3. ConfigMap data appears in plain text in etcd
4. ConfigMaps may be logged or exposed in events
5. Secrets can have separate, stricter RBAC policies

4.2 Encryption at Rest (EncryptionConfiguration)

By default, Secrets are stored **base64-encoded but NOT encrypted** in etcd. To encrypt them:

```
# /etc/kubernetes/enc/encryption-config.yaml
apiVersion: apiserver.config.k8s.io/v1
kind: EncryptionConfiguration
resources:
  - resources:
    - secrets
    providers:
    # Preferred provider (used for encryption)
    - aescbc:
        keys:
        - name: key1
          secret: <base64-encoded-32-byte-key>
    # Fallback provider (for reading unencrypted secrets)
    - identity: {}
```

```
Encryption Providers (strongest → weakest):
┌──────────────┬──────────────────────────────────────────┐
│ Provider     │ Description                              │
├──────────────┼──────────────────────────────────────────┤
│ kms (v2)     │ External KMS (AWS KMS, GCP KMS, Azure KV)  │
│              │ RECOMMENDED for production               │
├──────────────┼──────────────────────────────────────────┤
│ aescbc       │ AES-CBC with PKCS#7 padding              │
│              │ Strong, but key stored locally           │
├──────────────┼──────────────────────────────────────────┤
│ aesgcm       │ AES-GCM (must rotate keys frequently)    │
│              │ Fast, but nonce reuse risk               │
├──────────────┼──────────────────────────────────────────┤
│ secretbox    │ XSalsa20 + Poly1305                      │
│              │ Modern crypto, good performance          │
├──────────────┼──────────────────────────────────────────┤
│ identity     │ NO encryption (default)                  │
│              │ Plain base64 in etcd                     │
└──────────────┴──────────────────────────────────────────┘
```

Enable encryption on the API server:

```
# Add to kube-apiserver manifest
# /etc/kubernetes/manifests/kube-apiserver.yaml
spec:
  containers:
  - command:
    - kube-apiserver
    - --encryption-provider-config=/etc/kubernetes/enc/encryption-config.yaml
    # ... other flags
    volumeMounts:
    - name: enc-config
      mountPath: /etc/kubernetes/enc
      readOnly: true
  volumes:
  - name: enc-config
    hostPath:
      path: /etc/kubernetes/enc

# After enabling, encrypt all existing secrets:
kubectl get secrets --all-namespaces -o json | kubectl replace -f -
```

4.3 RBAC Policies for Secrets Access

```yaml
# Restrictive Role — only specific secrets, only in one namespace
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  namespace: production
  name: secret-reader
rules:
- apiGroups: [""]
  resources: ["secrets"]
  resourceNames: ["db-credentials", "api-keys"]  # Only specific secrets
  verbs: ["get"]                                  # No list, no watch

---
# Bind role to a specific service account
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: read-secrets
  namespace: production
subjects:
- kind: ServiceAccount
  name: app-service-account
  namespace: production
roleRef:
  kind: Role
  name: secret-reader
  apiGroup: rbac.authorization.k8s.io

---
# DENY list secrets (prevents seeing all secret names)
# This is done by NOT granting "list" verb on secrets
```

RBAC Best Practices:

```
┌──────────────────────────────────────────────────────────┐
│               Secret RBAC Checklist                      │
│                                                          │
│  □ Use resourceNames to restrict to specific secrets    │
│  □ Grant only "get", never "list" or "watch" unless needed │
│  □ Use namespaces to isolate secrets per environment    │
│  □ Audit who has access: kubectl auth can-i get secrets │
│  □ Regularly review RoleBindings                        │
│  □ Use ServiceAccounts, not user accounts, for apps     │
│  □ Never grant cluster-wide secret access to apps       │
│  □ Never grant "create" on secrets to untrusted users   │
└──────────────────────────────────────────────────────────┘
```

4.4 External Secret Managers

```
┌──────────────────────────────────────────────────────────────┐
│                  External Secret Management                  │
│                                                              │
│  ┌──────────────────┐      ┌──────────────────────────────┐  │
│  │ External Source  │      │   Kubernetes Operator/Controller │  │
│  │                  │──────>│                              │  │
│  │ • HashiCorp Vault│      │ • External Secrets Operator (ESO) │  │
│  │ • AWS Secrets Mgr│      │ • Vault Agent Injector       │  │
│  │ • Azure Key Vault│      │ • Sealed Secrets Controller  │  │
│  │ • GCP Secret Mgr │      │ • SOPS + age/gpg             │  │
│  └──────────────────┘      └──────────────┬───────────────┘  │
│                                           │                  │
│                                           ▼                  │
│                              ┌──────────────────┐            │
│                              │ Kubernetes Secret │            │
│                              │ (auto-synced)     │            │
│                              └──────────────────┘            │
└──────────────────────────────────────────────────────────────┘
```

HashiCorp Vault with Vault Agent Injector

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: app-pod
  annotations:
    vault.hashicorp.com/agent-inject: "true"
    vault.hashicorp.com/role: "myapp"
    vault.hashicorp.com/agent-inject-secret-db-creds: "secret/data/db-credentials"
    vault.hashicorp.com/agent-inject-template-db-creds: |
      {{- with secret "secret/data/db-credentials" -}}
      DB_USER={{ .Data.data.username }}
      DB_PASS={{ .Data.data.password }}
      {{- end }}
spec:
  serviceAccountName: myapp
  containers:
  - name: app
    image: myapp:1.0
    # Secret is available at /vault/secrets/db-creds
```

External Secrets Operator (ESO)

```yaml
apiVersion: external-secrets.io/v1beta1
kind: ExternalSecret
metadata:
  name: db-credentials
spec:
  refreshInterval: 1h              # Auto-rotation
  secretStoreRef:
    name: aws-secrets-manager
    kind: ClusterSecretStore
  target:
    name: db-credentials           # K8s Secret to create
    creationPolicy: Owner
  data:
  - secretKey: username
    remoteRef:
      key: prod/db-credentials
      property: username
  - secretKey: password
    remoteRef:
      key: prod/db-credentials
      property: password
```

Sealed Secrets (GitOps-Safe)

```bash
# Install kubeseal CLI
# Encrypt secret for the cluster (only the controller can decrypt)
kubectl create secret generic db-credentials \
  --from-literal=password='S3cur3P@ss!' \
  --dry-run=client -o yaml | kubeseal \
  --controller-name=sealed-secrets \
  --controller-namespace=kube-system \
  --format yaml > sealed-db-credentials.yaml
```

```yaml
# This is safe to commit to Git!
apiVersion: bitnami.com/v1alpha1
kind: SealedSecret
metadata:
  name: db-credentials
spec:
  encryptedData:
    password: AgBy3i4OJSWK+PiTySYZZA9rO43cGDEq...
```

4.5 Avoiding Secrets in Git (GitOps Considerations)

```
┌──────────────────────────────────────────────────────────────┐
│                    GitOps Secret Strategy                    │
│                                                              │
│   ┌──────────────────┐                                       │
│   │   Git Repo       │                                       │
│   │                  │                                       │
│   │ □ ConfigMaps     │          ┌──────────────────────┐     │
│   │ □ SealedSecrets  │ ──────> │ Sealed Secrets       │     │
│   │ □ ExternalSecrets│ ──────> │ Controller / ESO     │     │
│   │ □ SOPS-encrypted │ ──────> │ decrypts in-cluster  │     │
│   │                  │          └──────────┬───────────┘     │
│   │ □ Plain Secrets  │                     │                 │
│   │ □ .env files     │                     ▼                 │
│   │ □ Hardcoded keys │         ┌──────────────────┐          │
│   └──────────────────┘         │ K8s Secret (live)│          │
│                                └──────────────────┘          │
└──────────────────────────────────────────────────────────────┘
```

.gitignore for Kubernetes projects:

```
# Never commit plain secrets
*-secret.yaml
*-secrets.yaml
*.env
.env.*
**/secrets/

# Exception: sealed secrets ARE safe to commit
!*-sealed-secret.yaml
!*-sealedsecret.yaml
```

---

5. Hands-On Examples

5.1 kubectl CRUD Operations

ConfigMap Operations

```
# ── CREATE ──
kubectl create configmap app-config \
  --from-literal=APP_ENV=production \
  --from-literal=LOG_LEVEL=info

# ── READ ──
kubectl get configmaps                              # List all
kubectl get configmap app-config -o yaml            # Full YAML
kubectl describe configmap app-config               # Human-readable
kubectl get configmap app-config -o jsonpath='{.data.APP_ENV}'  # Single key

# ── UPDATE ──
kubectl edit configmap app-config                   # Opens in $EDITOR
kubectl patch configmap app-config \
  -p '{"data":{"LOG_LEVEL":"debug"}}'               # Patch single key

# Replace entirely from updated file
kubectl create configmap app-config \
  --from-file=config.yaml --dry-run=client -o yaml | kubectl apply -f -

# ── DELETE ──
kubectl delete configmap app-config
```

Secret Operations

```
# ── CREATE ──
kubectl create secret generic db-creds \
  --from-literal=username=admin \
  --from-literal=password='P@ssw0rd'

kubectl create secret tls tls-secret \
  --cert=tls.crt --key=tls.key

kubectl create secret docker-registry reg-creds \
  --docker-server=registry.example.com \
  --docker-username=user --docker-password=pass

# ── READ ──
kubectl get secrets                                  # List all
kubectl get secret db-creds -o yaml                  # Full YAML (base64)
kubectl describe secret db-creds                     # Shows keys, not values
kubectl get secret db-creds -o jsonpath='{.data.password}' | base64 -d  # Decode

# ── UPDATE ──
kubectl edit secret db-creds                         # Edit base64 values
kubectl patch secret db-creds \
  -p '{"stringData":{"password":"NewP@ss!"}}'        # Patch with plain text

# ── DELETE ──
kubectl delete secret db-creds
```

5.2 Complete ConfigMap YAML Manifest

```yaml
# configmap-complete-example.yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: myapp-config
  namespace: default
  labels:
    app: myapp
    tier: backend
  annotations:
    description: "Application configuration for myapp"
data:
  # Simple key-value pairs
  APP_ENV: "production"
  APP_PORT: "8080"
  LOG_LEVEL: "info"
  LOG_FORMAT: "json"
  CACHE_ENABLED: "true"
  CACHE_TTL: "300"

  # Database connection string (non-sensitive parts)
  DB_HOST: "postgres.database.svc.cluster.local"
  DB_PORT: "5432"
  DB_NAME: "myapp_production"
  DB_SSL_MODE: "require"

  # Full configuration file as a value
  config.yaml: |
    server:
      port: 8080
      host: 0.0.0.0
      read_timeout: 30s
      write_timeout: 30s
      idle_timeout: 120s
    database:
      max_open_conns: 25
      max_idle_conns: 5
      conn_max_lifetime: 5m
    cache:
      enabled: true
      ttl: 300
      max_entries: 10000
    features:
      new_dashboard: true
      beta_api: false
      dark_mode: true

  # Nginx config snippet
```

```
  nginx-proxy.conf: |
    upstream backend {
        server localhost:8080;
    }
    server {
        listen 80;
        location / {
            proxy_pass http://backend;
            proxy_set_header Host $host;
            proxy_set_header X-Real-IP $remote_addr;
            proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        }
        location /health {
            access_log off;
            return 200 "OK\n";
        }
    }
```

5.3 Complete Secret YAML Manifest

```yaml
# secret-complete-example.yaml
apiVersion: v1
kind: Secret
metadata:
  name: myapp-secrets
  namespace: default
  labels:
    app: myapp
    tier: backend
type: Opaque
stringData:
  # Database credentials
  DB_USERNAME: "myapp_user"
  DB_PASSWORD: "Sup3r$ecure#2024!"

  # API keys
  API_KEY: "sk-abc123def456ghi789"
  JWT_SECRET: "my-jwt-signing-secret-key-here"

  # External service credentials
  SMTP_PASSWORD: "email-service-password"
  AWS_SECRET_ACCESS_KEY: "wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY"

  # Full credentials file
  credentials.json: |
    {
      "type": "service_account",
      "project_id": "my-project",
      "private_key_id": "key-id-123",
      "private_key": "-----BEGIN RSA PRIVATE KEY-----\nMIIE...\n-----END RSA PRIVATE KEY-----\n",
      "client_email": "sa@my-project.iam.gserviceaccount.com"
    }
```

5.4 Sample Deployment Consuming Both ConfigMap and Secret

```yaml
# deployment-with-config-and-secrets.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: myapp
  namespace: default
  labels:
    app: myapp
spec:
  replicas: 3
  selector:
    matchLabels:
      app: myapp
  template:
    metadata:
      labels:
        app: myapp
      annotations:
        # Change this to trigger a rolling update when config changes
        configmap-version: "v1"
    spec:
      serviceAccountName: myapp-sa

      containers:
      - name: myapp
        image: myapp:2.0
        ports:
        - containerPort: 8080
          name: http

        # ── Environment Variables from ConfigMap ──
        envFrom:
        - configMapRef:
            name: myapp-config
          prefix: ""                    # No prefix

        # ── Individual Secret keys as env vars ──
        env:
        - name: DB_USERNAME
          valueFrom:
            secretKeyRef:
              name: myapp-secrets
              key: DB_USERNAME
        - name: DB_PASSWORD
          valueFrom:
            secretKeyRef:
              name: myapp-secrets
              key: DB_PASSWORD
```

```yaml
        - name: API_KEY
          valueFrom:
            secretKeyRef:
              name: myapp-secrets
              key: API_KEY
        - name: JWT_SECRET
          valueFrom:
            secretKeyRef:
              name: myapp-secrets
              key: JWT_SECRET

        # ── Volume Mounts ──
        volumeMounts:
        - name: config-files
          mountPath: /app/config
          readOnly: true
        - name: secret-files
          mountPath: /app/secrets
          readOnly: true
        - name: tls-certs
          mountPath: /etc/tls
          readOnly: true

        resources:
          requests:
            cpu: 100m
            memory: 128Mi
          limits:
            cpu: 500m
            memory: 512Mi

        livenessProbe:
          httpGet:
            path: /health
            port: http
          initialDelaySeconds: 10
        readinessProbe:
          httpGet:
            path: /ready
            port: http
          initialDelaySeconds: 5

      # ── Volumes ──
      volumes:
      # ConfigMap as volume (config files)
      - name: config-files
        configMap:
          name: myapp-config
```

```
        items:
        - key: config.yaml
          path: config.yaml
        - key: nginx-proxy.conf
          path: nginx.conf

    # Secret as volume (credential files)
    - name: secret-files
      secret:
        secretName: myapp-secrets
        defaultMode: 0400              # Read-only for owner
        items:
        - key: credentials.json
          path: gcp-credentials.json

    # TLS Secret as volume
    - name: tls-certs
      secret:
        secretName: myapp-tls-secret
        defaultMode: 0400
```

5.5 Projected Volumes — Multiple Sources in One Mount

```yaml
# projected-volume-example.yaml
apiVersion: v1
kind: Pod
metadata:
  name: app-with-projected-volume
spec:
  containers:
  - name: app
    image: myapp:1.0
    volumeMounts:
    - name: all-config
      mountPath: /etc/app-config
      readOnly: true
  volumes:
  - name: all-config
    projected:
      defaultMode: 0440
      sources:
      # Source 1: ConfigMap
      - configMap:
          name: app-settings
          items:
          - key: config.yaml
            path: config.yaml

      # Source 2: Secret
      - secret:
          name: app-credentials
          items:
          - key: db-password
            path: secrets/db-password

      # Source 3: Another ConfigMap
      - configMap:
          name: feature-flags
          items:
          - key: flags.json
            path: features/flags.json

      # Source 4: Downward API (pod metadata)
      - downwardAPI:
          items:
          - path: metadata/pod-name
            fieldRef:
              fieldPath: metadata.name
          - path: metadata/namespace
            fieldRef:
              fieldPath: metadata.namespace
```

```
          - path: metadata/labels
            fieldRef:
              fieldPath: metadata.labels

      # Source 5: ServiceAccount Token
      - serviceAccountToken:
          path: token
          expirationSeconds: 3600
          audience: "api.example.com"
```

Resulting directory structure inside the container:

```
/etc/app-config/
├── config.yaml                  (from ConfigMap: app-settings)
├── secrets/
│   └── db-password              (from Secret: app-credentials)
├── features/
│   └── flags.json               (from ConfigMap: feature-flags)
├── metadata/
│   ├── pod-name                 (from Downward API)
│   ├── namespace                (from Downward API)
│   └── labels                   (from Downward API)
└── token                        (from ServiceAccountToken)
```

---

6. Real-World Patterns

6.1 Managing Environment-Specific Configs (dev/staging/prod)

```
Project Structure:
├── base/
│   ├── deployment.yaml
│   ├── service.yaml
│   └── kustomization.yaml
├── overlays/
│   ├── dev/
│   │   ├── configmap.yaml       # Dev-specific config
│   │   ├── secret.yaml          # Dev secrets (or SealedSecret)
│   │   ├── kustomization.yaml
│   │   └── patches/
│   │       └── replicas.yaml    # 1 replica in dev
│   ├── staging/
│   │   ├── configmap.yaml
│   │   ├── sealed-secret.yaml
│   │   ├── kustomization.yaml
│   │   └── patches/
│   │       └── replicas.yaml    # 2 replicas in staging
│   └── prod/
│       ├── configmap.yaml
│       ├── sealed-secret.yaml
│       ├── kustomization.yaml
│       └── patches/
│           └── replicas.yaml    # 5 replicas in prod
```

Namespace-per-environment pattern:

```
# Create namespaces
kubectl create namespace dev
kubectl create namespace staging
kubectl create namespace production

# Deploy same ConfigMap with different values per namespace
kubectl create configmap app-config \
  --from-literal=APP_ENV=development \
  --from-literal=LOG_LEVEL=debug \
  --from-literal=DB_HOST=dev-db.internal \
  -n dev

kubectl create configmap app-config \
  --from-literal=APP_ENV=staging \
  --from-literal=LOG_LEVEL=info \
  --from-literal=DB_HOST=staging-db.internal \
  -n staging

kubectl create configmap app-config \
  --from-literal=APP_ENV=production \
  --from-literal=LOG_LEVEL=warn \
  --from-literal=DB_HOST=prod-db.internal \
  -n production
```

6.2 Using Kustomize Overlays for Config Management

Base Kustomization

```
# base/kustomization.yaml
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
resources:
  - deployment.yaml
  - service.yaml
```

```yaml
# base/deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: myapp
spec:
  selector:
    matchLabels:
      app: myapp
  template:
    metadata:
      labels:
        app: myapp
    spec:
      containers:
      - name: myapp
        image: myapp:latest
        envFrom:
        - configMapRef:
            name: myapp-config
        env:
        - name: DB_PASSWORD
          valueFrom:
            secretKeyRef:
              name: myapp-secrets
              key: DB_PASSWORD
```

Dev Overlay

```yaml
# overlays/dev/kustomization.yaml
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
namespace: dev
namePrefix: dev-
resources:
  - ../../base
configMapGenerator:
  - name: myapp-config
    literals:
      - APP_ENV=development
      - LOG_LEVEL=debug
      - DB_HOST=localhost
      - DB_PORT=5432
      - CACHE_ENABLED=false
secretGenerator:
  - name: myapp-secrets
    literals:
      - DB_PASSWORD=devpassword123
    # OR from file:
    # files:
    #   - secrets/db-password.txt
    type: Opaque
generatorOptions:
  disableNameSuffixHash: false  # Append hash for change detection
patches:
  - target:
      kind: Deployment
      name: myapp
    patch: |
      - op: replace
        path: /spec/replicas
        value: 1
```

Production Overlay

```yaml
# overlays/prod/kustomization.yaml
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
namespace: production
namePrefix: prod-
resources:
  - ../../base
  - sealed-secret.yaml          # Encrypted secret safe for Git
configMapGenerator:
  - name: myapp-config
    literals:
      - APP_ENV=production
      - LOG_LEVEL=warn
      - DB_HOST=prod-primary.database.svc.cluster.local
      - DB_PORT=5432
      - CACHE_ENABLED=true
      - CACHE_TTL=600
    # Also from files:
    files:
      - configs/nginx.conf
      - configs/application.yaml
generatorOptions:
  disableNameSuffixHash: false
patches:
  - target:
      kind: Deployment
      name: myapp
    patch: |
      - op: replace
        path: /spec/replicas
        value: 5
      - op: add
        path: /spec/template/spec/containers/0/resources
        value:
          requests:
            cpu: 500m
            memory: 512Mi
          limits:
            cpu: "2"
            memory: 2Gi
```

```
# Build and preview (dry-run)
kubectl kustomize overlays/dev
kubectl kustomize overlays/prod

# Apply
kubectl apply -k overlays/dev
kubectl apply -k overlays/prod
```

> ***CKA/CKAD Tip:*** *Kustomize is built into kubectl (`kubectl apply -k`). The `configMapGenerator` and `secretGenerator` automatically append a content hash suffix to names, which triggers a rolling update whenever config changes — solving the "env vars don't auto-update" problem.*

6.3 Helm Values vs ConfigMaps

```
┌──────────────────────────────────────────────────────────────┐
│                 Helm Values → ConfigMaps Pipeline            │
│                                                              │
│  values-dev.yaml ──┐                                         │
│  values-stg.yaml ──┼──> helm template ──> ConfigMap YAML ──> K8s  │
│  values-prd.yaml ──┘          │                              │
│                               │                              │
│                        templates/                            │
│                        configmap.yaml                        │
│                               │                              │
│  Chart.yaml ──────────────────┘                              │
└──────────────────────────────────────────────────────────────┘
```

Helm Chart ConfigMap Template

```
# templates/configmap.yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: {{ include "myapp.fullname" . }}-config
  labels:
    {{- include "myapp.labels" . | nindent 4 }}
data:
  APP_ENV: {{ .Values.app.env | quote }}
  LOG_LEVEL: {{ .Values.app.logLevel | quote }}
  DB_HOST: {{ .Values.database.host | quote }}
  DB_PORT: {{ .Values.database.port | quote }}
  {{- if .Values.app.configFile }}
  config.yaml: |
    {{- .Values.app.configFile | nindent 4 }}
  {{- end }}
```

```
# values-production.yaml
app:
  env: production
  logLevel: warn
  configFile: |
    server:
      port: 8080
    cache:
      enabled: true
      ttl: 600
database:
  host: prod-db.example.com
  port: "5432"
```

```
# Deploy with environment-specific values
helm install myapp ./chart -f values-production.yaml -n production

# Upgrade config
helm upgrade myapp ./chart -f values-production-v2.yaml -n production
```

Helm vs Kustomize for Config Management:

Aspect	Helm	Kustomize
Templating	Go templates (powerful, complex)	Patch-based (simple, declarative)
Config injection	values.yaml → templates	configMapGenerator + overlays
Secret handling	values + sealed-secrets	secretGenerator + sealed-secrets
Built into kubectl	No (separate install)	Yes (`kubectl apply -k`)
CKA/CKAD relevance	Less likely on exam	More likely on exam
Learning curve	Steeper	Gentler

6.4 Sidecar Pattern for Dynamic Config Reloading

```yaml
# sidecar-config-reload.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: myapp-with-reloader
spec:
  replicas: 3
  selector:
    matchLabels:
      app: myapp
  template:
    metadata:
      labels:
        app: myapp
    spec:
      containers:
      # ── Main Application Container ──
      - name: myapp
        image: myapp:2.0
        ports:
        - containerPort: 8080
        volumeMounts:
        - name: config
          mountPath: /app/config
          readOnly: true

      # ── Config Reloader Sidecar ──
      - name: config-reloader
        image: jimmidyson/configmap-reload:v0.9.0
        args:
          - --volume-dir=/config
          - --webhook-url=http://localhost:8080/-/reload
          - --webhook-method=POST
        volumeMounts:
        - name: config
          mountPath: /config
          readOnly: true
        resources:
          requests:
            cpu: 10m
            memory: 16Mi
          limits:
            cpu: 50m
            memory: 32Mi

      volumes:
      - name: config
```

```
        configMap:
          name: myapp-config
```

```
Flow:
┌──────────────┐     ┌──────────────────┐     ┌──────────────┐
│  ConfigMap   │────>│  Volume Mount    │────>│  Sidecar     │
│  (updated)   │     │  (auto-synced    │     │  (watches    │
│              │     │   by kubelet)    │     │   for change)│
└──────────────┘     └──────────────────┘     └──────┬───────┘
                                                      │
                                                webhook POST
                                                /-/reload
                                                      │
                                                      ▼
                                                ┌──────────────┐
                                                │  Main App    │
                                                │  (reloads    │
                                                │   config)    │
                                                └──────────────┘
```

Alternative: Stakater Reloader (Operator-based approach)

```
# Instead of a sidecar, install Reloader as a cluster-wide operator
# It watches ConfigMap/Secret changes and triggers rolling updates

apiVersion: apps/v1
kind: Deployment
metadata:
  name: myapp
  annotations:
    # Reloader watches this ConfigMap and restarts pods on change
    configmap.reloader.stakater.com/reload: "myapp-config"
    # Also watch secrets
    secret.reloader.stakater.com/reload: "myapp-secrets"
spec:
  # ... normal deployment spec
```

---

7. CKA/CKAD Exam Tips

7.1 Speed Commands (Exam Time-Savers)

```
# ── Aliases (add to ~/.bashrc at exam start) ──
alias k=kubectl
alias kgcm='kubectl get configmap'
alias kgsec='kubectl get secret'

# Enable autocomplete
source <(kubectl completion bash)
complete -o default -F __start_kubectl k

# ── Quick ConfigMap creation ──
k create cm my-config --from-literal=key=value --dry-run=client -o yaml > cm.yaml

# ── Quick Secret creation ──
k create secret generic my-secret --from-literal=pass=secret --dry-run=client -o yaml > sec.yaml

# ── Quick decode ──
k get secret my-secret -o jsonpath='{.data.pass}' | base64 -d; echo

# ── Quick volume mount check ──
k exec -it pod-name -- ls /path/to/mount
k exec -it pod-name -- cat /path/to/mount/key-name
k exec -it pod-name -- env | grep KEY_NAME
```

7.2 Common Exam Scenarios

```
┌──────────────────────────────────────────────────────────────────┐
│              CKA/CKAD ConfigMap/Secret Scenarios                 │
│                                                                  │
│  1. Create ConfigMap from literals/files   →  kubectl create cm  │
│  2. Create Secret from literals/files      →  kubectl create secret │
│  3. Mount ConfigMap as volume in Pod       →  volumes + volumeMounts │
│  4. Inject Secret as env var               →  env + secretKeyRef  │
│  5. Use envFrom with ConfigMap             →  envFrom + configMapRef │
│  6. Create immutable ConfigMap             →  immutable: true     │
│  7. Decode a secret value                  →  base64 --decode     │
│  8. Use projected volumes                  →  projected.sources   │
│  9. Debug: "why isn't my config updating?" →  Check subPath usage │
│ 10. Security: encrypt secrets at rest      →  EncryptionConfiguration│
└──────────────────────────────────────────────────────────────────┘
```

7.3 Common Pitfalls to Avoid

Pitfall	Correct Approach
Storing API keys in ConfigMaps	Always use Secrets for sensitive data
Expecting env vars to auto-update	Env vars require pod restart; use volume mounts for live updates
Using `subPath` and expecting updates	`subPath` mounts do NOT receive updates

Pitfall	Correct Approach
Forgetting `base64 -d` for secrets	Always decode when reading secret values
Secrets in wrong namespace	ConfigMaps and Secrets must be in the same namespace as the Pod
Missing `echo -n` when creating files	Trailing newline breaks passwords/tokens
Large files in ConfigMaps	Max 1 MiB; use Persistent Volumes for larger files
Not setting `readOnly: true` on secret mounts	Always mount secrets as read-only
Committing secrets to Git	Use Sealed Secrets, ESO, or SOPS
Not setting restrictive file permissions	Use `defaultMode: 0400` for secret volumes

7.4 Quick Reference Card

```
┌──────────────────────────────────────────────────────────────────────┐
│                    CONFIGMAP & SECRET QUICK REFERENCE                 │
├──────────────────────────────────────────────────────────────────────┤
│                                                                      │
│  CREATE:                                                             │
│    ConfigMap:  kubectl create cm NAME --from-literal=k=v             │
│    Secret:     kubectl create secret generic NAME --from-literal=k=v │
│    TLS:        kubectl create secret tls NAME --cert=X --key=Y       │
│    Registry:   kubectl create secret docker-registry NAME --docker-* │
│                                                                      │
│  READ:                                                               │
│    kubectl get cm/secret NAME -o yaml                                │
│    kubectl describe cm/secret NAME                                   │
│    kubectl get secret NAME -o jsonpath='{.data.KEY}' | base64 -d     │
│                                                                      │
│  CONSUME AS ENV:                                                     │
│    configMapKeyRef / secretKeyRef  (single key)                      │
│    configMapRef / secretRef        (all keys via envFrom)            │
│                                                                      │
│  CONSUME AS VOLUME:                                                  │
│    volumes:                                                          │
│    - name: vol                                                       │
│      configMap:                    # or secret:                      │
│        name: NAME                  #    secretName: NAME             │
│        items:                      #    items:                       │
│        - key: k                    #    - key: k                     │
│          path: filename            #      path: filename             │
│    containers:                                                       │
│    - volumeMounts:                                                   │
│      - name: vol                                                     │
│        mountPath: /path                                              │
│        readOnly: true                                                │
│                                                                      │
│  PROJECTED VOLUME:                                                   │
│    volumes:                                                          │
│    - name: vol                                                       │
│      projected:                                                      │
│        sources:                                                      │
│        - configMap: {name: X}                                        │
│        - secret: {name: Y}                                           │
│        - downwardAPI: {items: [...]}                                 │
│        - serviceAccountToken: {path: token, expirationSeconds: 3600} │
│                                                                      │
│  IMMUTABLE:  Add "immutable: true" to ConfigMap/Secret spec          │
│                                                                      │
│  KEY FACTS:                                                          │
│    • Max size: 1 MiB                                                 │
│    • Namespace-scoped                                                │
```

```
│    • Env vars don't auto-update; volume mounts do (~60-90s)             │
│    • subPath mounts do NOT auto-update                                  │
│    • Secrets use tmpfs (in-memory) for volume mounts                    │
│    • Secrets are base64-encoded, NOT encrypted (by default)             │
│                                                                         │
└─────────────────────────────────────────────────────────────────────────┘
```

Appendix A: Correcting the Project's Anti-Pattern

The existing file `openai-api-config.yaml` in this project stores an API key in a ConfigMap:

```
# □ WRONG — openai-api-config.yaml (current)
apiVersion: v1
kind: ConfigMap
metadata:
  name: openai-api-config
data:
  OPEN_AI_KEY: sk-Shdn30x=3n0920nshd3930djdhwjJGD0dnHdl(hdld93h
```

This should be refactored to a Secret:

```
# □ CORRECT — openai-api-secret.yaml
apiVersion: v1
kind: Secret
metadata:
  name: openai-api-secret
type: Opaque
stringData:
  OPEN_AI_KEY: sk-Shdn30x=3n0920nshd3930djdhwjJGD0dnHdl(hdld93h
```

And consumed in a Deployment:

```
env:
- name: OPEN_AI_KEY
  valueFrom:
    secretKeyRef:
      name: openai-api-secret
      key: OPEN_AI_KEY
```

End of Study Notes — Last updated: 2026-03-30